APUNTES. ACCESO A ARCHIVOS

Vamos a ver en este tema como acceder a archivos, y guardar información para darle persistencia a nuestros datos.

Hasta ahora, al acabar el programa, perdemos los datos, ya que se libera la memoria RAM y se borran los datos que están en ella al finalizar el programa, dado que se liberan los recursos de la misma.

Eso acaba con la persistencia. Guardaremos la información importante en ficheros de modo que los podamos recuperar, leer y escribir información en ellos para usarla a nuestro gusto.

Apuntes de C#: Acceso a archivos (File I/O)

Leer, escribir y añadir enteros y otra información en ficheros

1. Idea clave: un archivo es un flujo de datos

En C#, cuando trabajamos con archivos estamos leyendo y escribiendo datos en disco. Lo habitual es usar clases del espacio de nombres System.IO. Las operaciones básicas son:

• Crear/abrir un fichero
• Escribir datos
• Cerrar el fichero (liberar recursos)
• Abrir de nuevo para leer
• (Opcional) añadir datos sin borrar lo que ya hay

2. Recomendación práctica para principiantes: guardar enteros en texto

Para empezar, lo más sencillo es guardar los enteros en un archivo de texto, uno por línea. Así es fácil de inspeccionar con un editor de texto y fácil de depurar.

- Archivo de texto: legible por humanos (ej.: 10↵20↵30↵).
- Archivo binario: más compacto, pero no legible (se usa en escenarios más avanzados).

3. Cerrar el archivo: using (lo más importante)

En C#, la forma recomendada de abrir un archivo es usando la palabra clave using. using asegura que el archivo se cierra automáticamente, incluso si ocurre un error.

```
using (var writer = new StreamWriter("datos.txt"))
{
    writer.WriteLine("Hola");
} // aquí se cierra automáticamente
```

4. Ejemplo completo: crear un fichero, guardar enteros y cerrarlo

Objetivo: el programa crea un fichero, guarda varios enteros y lo cierra.
En este ejemplo guardamos 5 enteros (uno por línea).

```
using System;
using System.IO;

class Program
{
    static void Main()
    {
        string ruta = "enteros.txt"; // se crea en la carpeta del ejecutable
(bin/Debug/...)

        int[] valores = { 10, 20, 30, 40, 50 };

        using (StreamWriter writer = new StreamWriter(ruta))
        {
            foreach (int v in valores)
            {
                writer.WriteLine(v);
            }
        } // cerrado automático

        Console.WriteLine($"Archivo creado y guardado:
{Path.GetFullPath(ruta)}");
    }
}
```

5. Leer el archivo: recuperar los enteros y cerrarlo

Ahora abrimos el mismo archivo para lectura. Como son enteros en texto, leemos línea a línea y convertimos (Parse/TryParse).

```
using System;
using System.IO;

class Program
{
```

```
    static void Main()
    {
        string ruta = "enteros.txt";

        if (!File.Exists(ruta))
        {
            Console.WriteLine("El archivo no existe.");
            return;
        }

        using (StreamReader reader = new StreamReader(ruta))
        {
            string linea;
            while ((linea = reader.ReadLine()) != null)
            {
                if (int.TryParse(linea, out int numero))
                    Console.WriteLine($"Leído: {numero}");
                else
                    Console.WriteLine($"Línea no válida: {linea}");
            }
        } // cerrado automático
    }
}
```

6. Añadir valores al archivo (append) sin borrar el contenido

Por defecto, StreamWriter sobrescribe el archivo. Para AÑADIR al final, usa el constructor con el parámetro append=true.

```
using System;
using System.IO;

class Program
{
    static void Main()
    {
        string ruta = "enteros.txt";

        int[] nuevos = { 60, 70, 80 };

        using (StreamWriter writer = new StreamWriter(ruta, append: true))
        {
            foreach (int v in nuevos)
            {
                writer.WriteLine(v);
            }
        }
```

```
        Console.WriteLine("Valores añadidos al final del archivo.");
    }
}
```

7. Alternativa rápida: File.WriteAllLines / File.ReadAllLines

Para ejemplos sencillos, la clase File permite escribir y leer con una sola llamada. Ojo: ReadAllLines carga TODO el archivo a memoria, útil para archivos pequeños.

```
using System;
using System.IO;
using System.Linq;

class Program
{
    static void Main()
    {
        string ruta = "enteros.txt";

        // Escribir (sobrescribe)
        int[] datos = { 1, 2, 3 };
        File.WriteAllLines(ruta, datos.Select(x => x.ToString()));

        // Leer (carga todo)
        string[] lineas = File.ReadAllLines(ruta);
        int[] enteros = lineas.Select(l => int.Parse(l)).ToArray();

        Console.WriteLine(string.Join(", ", enteros));
    }
}
```

8. Buenas prácticas para clase (muy recomendables)

- Usa rutas claras: Path.Combine(carpeta, nombreArchivo).
- Comprueba File.Exists antes de leer.
- Usa int.TryParse para evitar que el programa se caiga por datos corruptos.
- Explica la diferencia entre sobrescribir y añadir (append).
- Mantén el formato del fichero simple: 1 número por línea.

9. Control de rutas (carpetas) y Path.Combine

Si quieres guardar el archivo en una carpeta concreta, combina rutas de forma segura con Path.Combine.

```
string carpeta =
Environment.GetFolderPath(Environment.SpecialFolder.Desktop);
string ruta = Path.Combine(carpeta, "enteros.txt");
Console.WriteLine(ruta);
```

10. (Opcional) Guardar enteros en binario: BinaryWriter / BinaryReader

Si quieres mostrar una alternativa más avanzada: en binario el archivo ocupa menos y la lectura/escritura es directa, pero NO es legible con un editor de texto. Se usa mucho en escenarios de rendimiento.

```
using System;
using System.IO;

class Program
{
    static void Main()
    {
        string ruta = "enteros.bin";
        int[] datos = { 10, 20, 30 };

        // ESCRIBIR BINARIO
        using (var bw = new BinaryWriter(File.Open(ruta, FileMode.Create)))
        {
            bw.Write(datos.Length); // guardamos primero cuántos hay
            foreach (int n in datos) bw.Write(n);
        }

        // LEER BINARIO
        using (var br = new BinaryReader(File.Open(ruta, FileMode.Open)))
        {
            int cantidad = br.ReadInt32();
            for (int i = 0; i < cantidad; i++)
            {
                int n = br.ReadInt32();
                Console.WriteLine(n);
            }
        }
    }
}
```

11. Mini-proyecto para alumnos (Resumen)

Propuesta: programa de consola con un menú:

1) Guardar enteros (sobrescribir)

2) Leer enteros

3) Añadir enteros

0) Salir

Practicar arrays, bucles, switch, File/StreamWriter/StreamReader, validación y rutas.